Cryptography Lib Lab Project Report

Path 2: AES vs DES Algorithm Comparison

Done By: Marwa Hussein Ahmed Ali

Computers And Systems Department

Supervised by: Dr. Ahmed Hussein

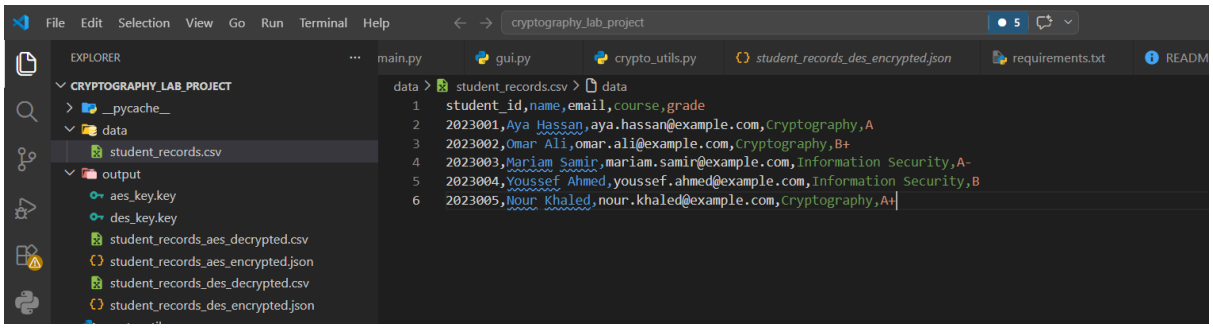
Academic Term: Second Semester

Year: 2026

Date : 14/5/2026

1. Project Scenario and Overview

This project is a Cryptography Lib Lab application based on **Path 2: Algorithm Comparison**. The selected scenario is **Secure Student Records**, where a CSV file containing fake student records is encrypted and decrypted. The project uses real-format data from data/student_records.csv, not just a simple word or hard-coded plaintext. The main goal is to protect the student records file by converting the readable CSV plaintext into unreadable ciphertext, then decrypting it again and proving that the recovered file is exactly the same as the original file. This matches the lab requirement to use real data, encrypt it, save the encrypted result, decrypt it, and verify successful recovery.



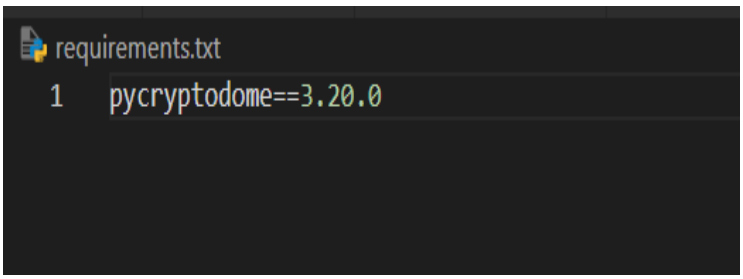
2. Selected Path

I selected **Path 2: Algorithm Comparison**. In this path, the same input data must be encrypted using both **AES** and **DES**, then the results must be compared using ciphertext output, execution time, key size, block size, and security notes. AES and DES were chosen because the project instructions specifically list AES and DES as the algorithms required for Path 2.

Path 2: Algorithm Comparison	AES and DES	Encrypt the same data using AES and DES. Compare ciphertext output, execution time, key size, block size, and security notes.	Medium

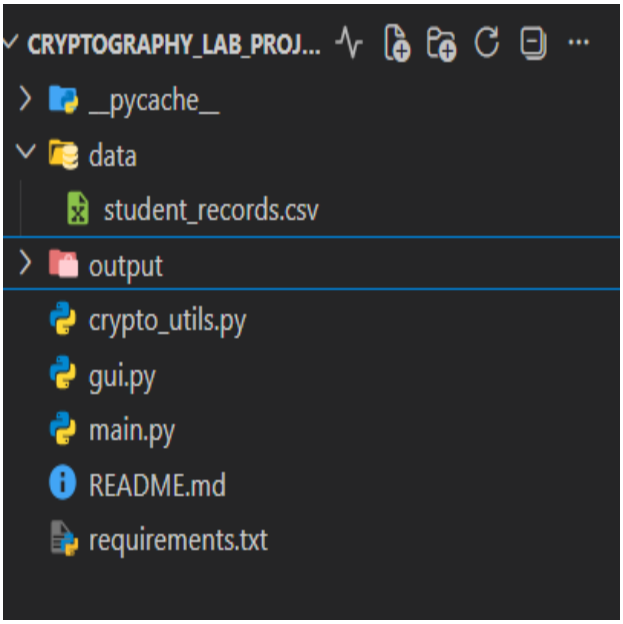
3. Programming Language and Library

The project was implemented in **Python** using the **PyCryptodome** library. The requirements.txt file contains pycryptodome==3.20.0, which means the project uses a trusted cryptography library instead of manually writing the encryption algorithms from scratch.



4. Project Files and Structure

The project is organized into several files. The main terminal version is in main.py, the encryption and decryption functions are in crypto_utils.py, and the optional graphical interface is in gui.py. The input data file is stored as data/student_records.csv, while encrypted files, decrypted files, and key files are saved inside the output folder. The README describes the project as Path 2, using the Secure Student Records scenario, AES-256-GCM, DES-CBC, SHA-256 verification, Base64 output, execution-time comparison, and a Tkinter GUI.



5. Input Data

The input file used in this project is student_records.csv. It contains fake student data such as student IDs, names, emails, courses, and grades. This is suitable for the project because the lab requires real-format data, such as a CSV file containing student names, IDs, grades, or emails. The data is fake, so it does not expose private real personal information.

data > student_records.csv > data

```
1  student_id,name,email,course,grade
2  2023001,Aya Hassan,aya.hassan@example.com,Cryptography,A
3  2023002,Omar Ali,omar.ali@example.com,Cryptography,B+
4  2023003,Mariam Samir,mariam.samir@example.com,Information Security,A-
5  2023004,Youssef Ahmed,youssef.ahmed@example.com,Information Security,B
6  2023005,Nour Khaled,nour.khaled@example.com,Cryptography,A+
```

6. AES Implementation

The first algorithm used is **AES-256-GCM**. AES uses a **256-bit key**, stored as 32 random bytes. In this project, AES-GCM produces the ciphertext, a nonce, and an authentication tag. AES-GCM does not need padding, which means the ciphertext size can be the same as the plaintext size. The encrypted AES result is saved as a JSON file containing the algorithm name, nonce in Base64, tag in Base64, ciphertext in Base64, plaintext size, ciphertext size, key size, block size, and a security note.

```
91  def encrypt_with_aes(input_path, output_json_path, key_path):
92      """
93      Encrypt the input file using AES-256-GCM.
94
95      AES-GCM does not need padding.
96      It produces:
97      - ciphertext
98      - nonce
99      - authentication tag
100     """
101     plaintext = read_file_bytes(input_path)
102
103     key = get_random_bytes(AES_KEY_SIZE_BYTES)
104     save_key(key_path, key)
105
106     cipher = AES.new(key, AES.MODE_GCM)
107
108     start_time = time.perf_counter()
109     ciphertext, tag = cipher.encrypt_and_digest(plaintext)
110     encryption_time = time.perf_counter() - start_time
111
112     encrypted_package = {
113         "algorithm": "AES-256-GCM",
114         "nonce_base64": to_base64(cipher.nonce),
115         "tag_base64": to_base64(tag),
116         "ciphertext_base64": to_base64(ciphertext),
117         "plaintext_size_bytes": len(plaintext),
118         "ciphertext_size_bytes": len(ciphertext),
119         "key_size_bits": AES_KEY_SIZE_BYTES * 8,
120         "block_size_bits": AES_BLOCK_SIZE_BYTES * 8,
121         "security_note": "AES is a modern recommended symmetric encryption algorithm."
122     }
123
```

7. DES Implementation

The second algorithm used is **DES-CBC**. DES uses an 8-byte key, but its effective security strength is only 56 bits. DES-CBC also uses an 8-byte IV and needs PKCS#7 padding because DES works on fixed-size blocks. The encrypted DES result is also saved as a JSON file containing the algorithm name, IV in Base64, ciphertext in Base64, plaintext size, ciphertext size, key size, block size, and a security note explaining that DES is weak and used only for educational comparison.

```python
def encrypt_with_des(input_path, output_json_path, key_path):
    """
    Encrypt the input file using DES-CBC.

    DES-CBC needs:
    - 8-byte key
    - 8-byte IV
    - PKCS#7 padding because DES works on fixed-size blocks
    """
    plaintext = read_file_bytes(input_path)

    key = get_random_bytes(DES_KEY_SIZE_BYTES)
    save_key(key_path, key)

    iv = get_random_bytes(DES_BLOCK_SIZE_BYTES)

    cipher = DES.new(key, DES.MODE_CBC, iv=iv)
    padded_plaintext = pad(plaintext, DES_BLOCK_SIZE_BYTES)

    start_time = time.perf_counter()
    ciphertext = cipher.encrypt(padded_plaintext)
    encryption_time = time.perf_counter() - start_time

    encrypted_package = {
        "algorithm": "DES-CBC",
        "iv_base64": to_base64(iv),
        "ciphertext_base64": to_base64(ciphertext),
        "plaintext_size_bytes": len(plaintext),
        "ciphertext_size_bytes": len(ciphertext),
        "key_size_bits": 56,
        "block_size_bits": DES_BLOCK_SIZE_BYTES * 8,
        "security_note": "DES is weak and allowed here only for educational comparison."
    }
```

8. Program Flow

The program follows the required cryptography flow. First, it loads the original CSV file. Then it generates encryption keys, encrypts the same file using AES and DES, saves the encrypted outputs, decrypts both encrypted files, and finally compares the decrypted files with the original file.

```
============================================================
CRYPTOGRAPHY LIB LAB MENU - PATH 2
============================================================
1. Preview original file
2. Encrypt using AES
3. Decrypt AES output
4. Encrypt using DES
5. Decrypt DES output
6. Verify decrypted files
7. Run full AES vs DES comparison
0. Exit
Choose an option:
```

The terminal program provides a menu with options to preview the original file, encrypt using AES, decrypt AES output, encrypt using DES, decrypt DES output, verify files, or run the full comparison demo.

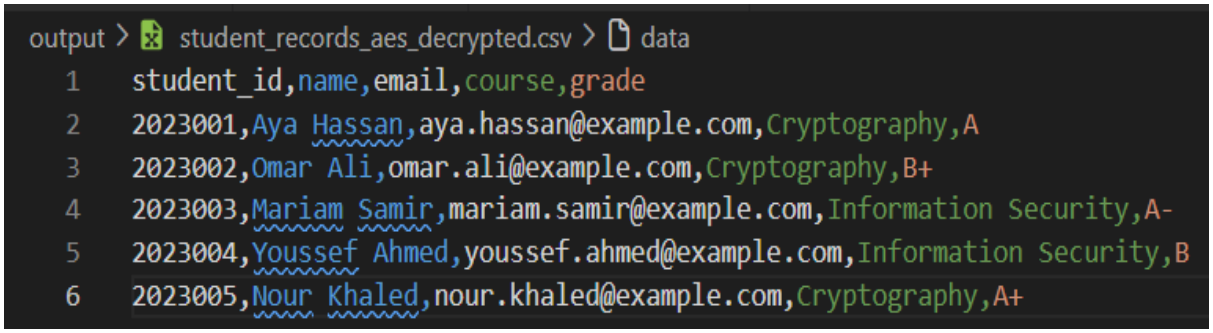
---

9. Encrypted Output

The encrypted output is not readable like the original CSV file. Instead of normal names, emails, and grades, the program saves random-looking Base64 ciphertext inside JSON files. For AES, the file is saved as **output/student_records_aes_encrypted.json**. For DES, the file is saved as **output/student_records_des_encrypted.json**. Base64 is used because encrypted binary data is difficult to display directly, so Base64 makes it easier to save and view in a text-based JSON file.

10. Decrypted Output

After encryption, the program decrypts the AES and DES ciphertext files. The AES decrypted file is saved as output/student_records_aes_decrypted.csv, and the DES decrypted file is saved as output/student_records_des_decrypted.csv. Both decrypted files should become readable CSV files again, with the same student records as the original input file. The program writes the recovered plaintext bytes back to CSV files after decryption.



output > student_records_des_decrypted.csv > data

```
student_id,name,email,course,grade
2023001,Aya Hassan,aya.hassan@example.com,Cryptography,A
2023002,Omar Ali,omar.ali@example.com,Cryptography,B+
2023003,Mariam Samir,mariam.samir@example.com,Information Security,A-
2023004,Youssef Ahmed,youssef.ahmed@example.com,Information Security,B
2023005,Nour Khaled,nour.khaled@example.com,Cryptography,A+
```

11. Comparison Results

In my sample run, the original CSV file size was **351 bytes**. AES-GCM produced a ciphertext size of **351 bytes**, while DES-CBC produced a ciphertext size of **352 bytes**. DES output became slightly larger because DES-CBC uses padding, while AES-GCM does not require padding. In the same sample run, AES encryption time was about **0.00011718 seconds**, and DES encryption time was about **0.00004781 seconds**. AES decryption time was about **0.00056377 seconds**, and DES decryption time was about **0.00007388 seconds**. These time values can change depending on the computer and each program run.

```
Now decrypting AES and DES outputs.

============================================================
AES DECRYPTION
============================================================
AES decrypted file saved: output/student_records_aes_decrypted.csv
AES decryption time: 0.00038850 seconds

============================================================
DES DECRYPTION
============================================================
DES decrypted file saved: output/student_records_des_decrypted.csv
DES decryption time: 0.00005090 seconds

============================================================
SHA-256 VERIFICATION
============================================================
AES original hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
AES decrypted hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
AES verification result: SUCCESS - decrypted file matches original.

DES original hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
DES decrypted hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
DES verification result: SUCCESS - decrypted file matches original.
```

12. AES vs DES Comparison Table

Feature	AES-256-GCM	DES-CBC
Key Size	256 bits	56 effective bits
Block Size	128 bits	64 bits
Mode	GCM	CBC
IV/Nonce	Nonce	IV
Padding	Not needed	PKCS#7 padding
Sample Ciphertext Size	351 bytes	352 bytes
Security	Modern and recommended	Weak, educational only

AES is the stronger and recommended algorithm because it uses a much larger key size and is considered modern. DES is included only to compare it with AES and to understand why older algorithms are no longer suitable for real secure applications. The lab instructions also mention that DES is allowed for educational comparison only, while AES is recommended for modern secure applications.

```
============================================================
COMPARISON SUMMARY
============================================================
AES:
- Algorithm: AES-256-GCM
- Key size: 256 bits
- Block size: 128 bits
- Padding: Not needed in GCM mode
- Security: Modern and recommended

DES:
- Algorithm: DES-CBC
- Key size: 56 effective bits
- Block size: 64 bits
- Padding: PKCS#7 padding used
- Security: Weak and used here only for educational comparison
```

13. Verification Method

To prove that decryption worked correctly, the program uses **SHA-256 hashing**. It calculates the SHA-256 hash of the original CSV file and compares it with the SHA-256 hash of each decrypted file. If the hashes are identical, the files are exactly the same. In my sample run, the original hash and both decrypted hashes matched:

c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159

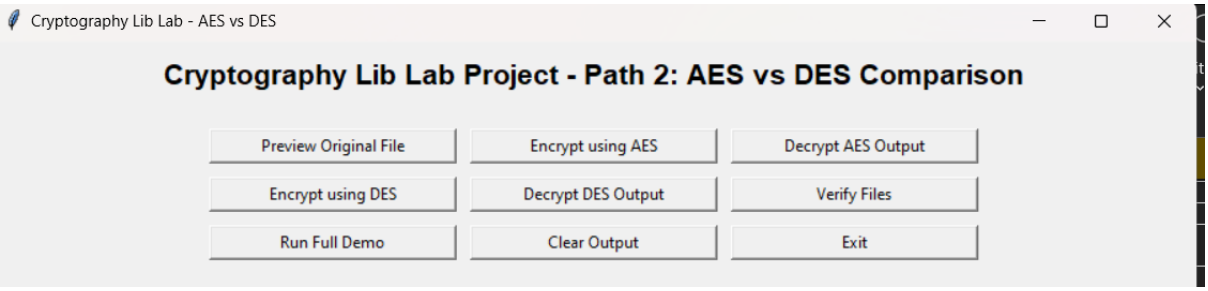
The program printed: **AES verification result: SUCCESS - decrypted file matches original** and **DES verification result: SUCCESS - decrypted file matches original**. This satisfies the requirement to prove that the decrypted output is exactly the same as the original input.

```
============================================================
SHA-256 VERIFICATION
============================================================
AES original hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
AES decrypted hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
AES verification result: SUCCESS - decrypted file matches original.

DES original hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
DES decrypted hash:
c57183e7ceafd9e4c69f3c7c02ff10bbd7b74fcb435c79ef5c6c1216e520b159
DES verification result: SUCCESS - decrypted file matches original.
```

14. GUI Extra Feature

The project also includes a simple **Tkinter GUI** in gui.py. The GUI contains buttons to preview the original file, encrypt using AES, decrypt AES output, encrypt using DES, decrypt DES output, verify files, run the full demo, clear output, and exit. This is an extra feature because the lab document lists a simple GUI as an acceptable extra feature.



15. Security Discussion

This project uses **AES-256-GCM** and **DES-CBC** because Path 2 requires comparing AES and DES on the same data. AES is the modern and recommended algorithm, while DES **is weak and used only for educational comparison**. The encryption key is the secret value used to encrypt and decrypt the file; **AES uses a 256-bit key**, while DES uses a **56-bit effective key**. AES-GCM uses a nonce, and **DES-CBC** uses an IV to add randomness to the encryption process so the ciphertext is not predictable. The encrypted output looks completely different from the original CSV because it appears as unreadable Base64 ciphertext inside JSON files. To prove that decryption worked correctly, the program compares the **SHA-256 hash** of the original file with the SHA-256 hash of the decrypted files. **One security limitation** is that the key files are saved locally in the output folder, which is acceptable for this lab but not secure for a real system.

16. Conclusion

This project successfully demonstrates how a real CSV student records file can be encrypted and decrypted using a cryptography library. AES-256-GCM and DES-CBC were applied to the same input file, and the results were compared by ciphertext size, execution time, key size, block size, padding, and security level. The decrypted files were verified using SHA-256 hashes, and both AES and DES recovered the original data correctly. The comparison shows that AES is the better and more secure modern choice, while DES is useful only for educational comparison.